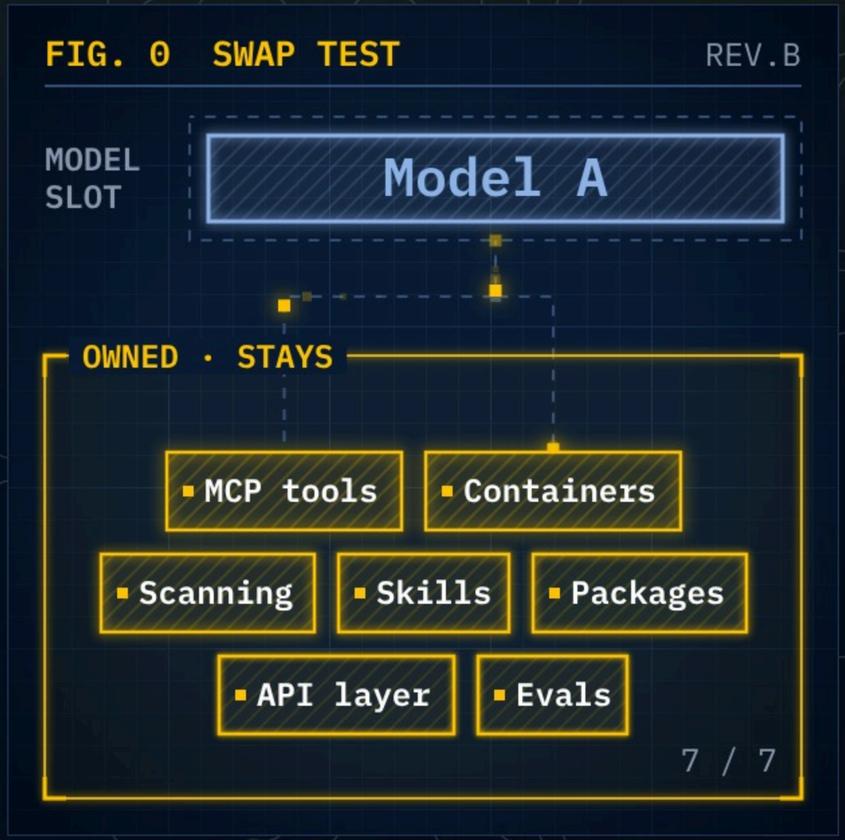
Kinvecta

TRAIN-THE-TRAINER SERIES · SESSION 2 OF 8

AI resilience: own what doesn't change

Rent the model. Own the connections, the know-how and the proof.



9-minute video

Foundation · before Stage 1

Guide Part 3.1

Kinvecta

TODAY

By the end, your learners can…

1
Explain what makes an organization agnostic

2
Name the seven foundations it owns

3
Show how a model switch becomes a config change

4
Run a lock-in audit on a real workflow

CHAPTERS · 9 MIN

drawn to scale



Kinvecta

WHY RESILIENCE

The model you use today will not be the one you use next year

01

Models retire

Versions are replaced and older ones are switched off on the vendor's schedule, not yours.

02

Terms change

Prices, data handling, rate limits and contracts move, sometimes with little notice.

03

Approvals change

What security authorizes today can change in either direction: a tool added, a tool withdrawn.

The question for this session:

What do we own that survives a switch?

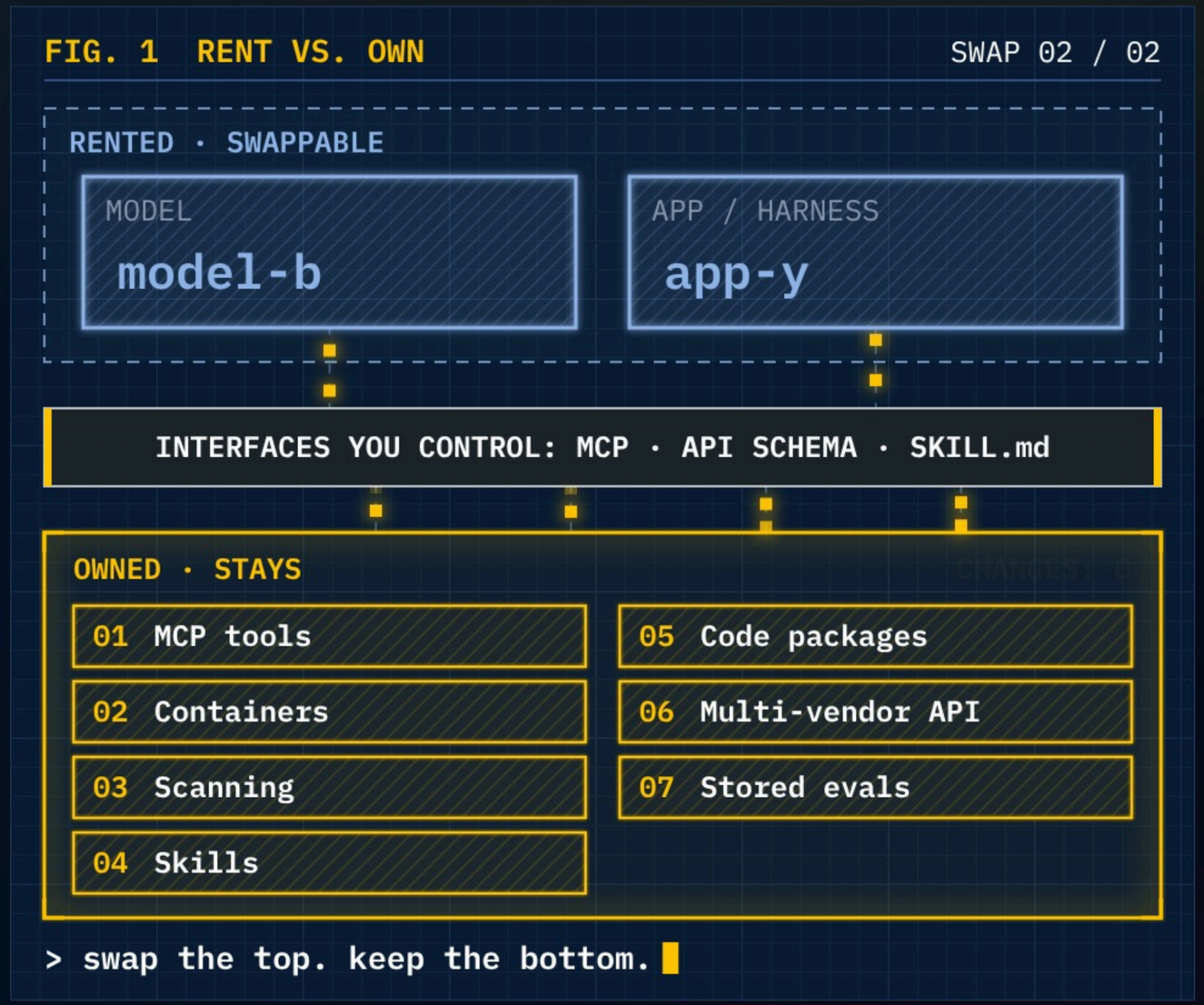
Kinvecta

WHAT AGNOSTIC MEANS

Swap the model, or the whole vendor, without rebuilding the work

Rent
Models, vendor apps and harnesses: expect them to change

Own
Seven foundations: the parts every use case runs on



Kinvecta

FOUR KINDS OF SWITCH

A switch can happen at four levels

MODEL VERSION

Same vendor, newer model

`model-a-05 → model-a-09`

Holds if: prompts and evals are stored

VENDOR

A different model family

`Vendor A → Vendor B`

Holds if: the workflow layer is vendor-neutral

ENDPOINT

Same model, different door

`vendor API → cloud → platform`

Holds if: the endpoint lives in config

HARNESS

A different app or agent tool

`one harness → another`

Holds if: MCP tools and skills are portable

Kinvecta

WHAT AGNOSTIC IS NOT

Four beliefs to correct early

BELIEF

"Agnostic means using every vendor"

REALITY

It means being able to switch, usually while running one

BELIEF

"A gateway makes us agnostic"

REALITY

It helps; prompts, tools, skills and evals must be portable too

BELIEF

"Switching is a migration project"

REALITY

With config and evals it is a tested, one-line change

BELIEF

"Agnostic means lowest common denominator"

REALITY

Use vendor features, behind interfaces you own

Kinvecta

FIGURE 13 IN THE GUIDE

Seven foundations, one loop

In order

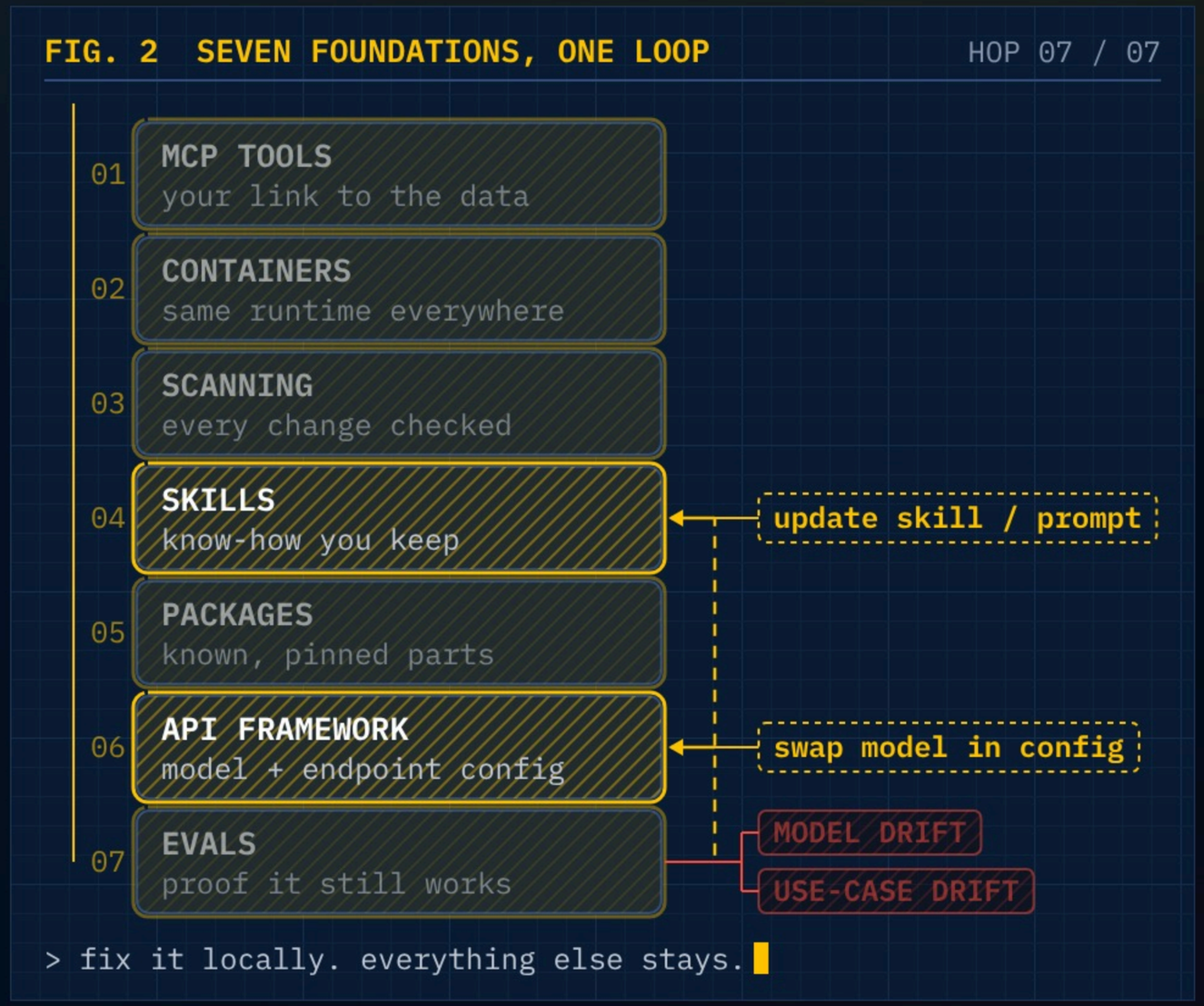
A use case meets each foundation, top to bottom, and ends in evals

Two kinds of drift

The model changes, or the use case changes under the same prompt

Fix it locally

Swap the model in config, or update a skill or prompt; everything else stays



Kinvecta

SEVEN THINGS YOU OWN

None of them belongs to a model vendor

01

MCP tools

Your connection to the data; any MCP harness can use it

02

Containers

The authorization attaches to the image, not the model

03

Scanning

Every new package, skill or model release gets the same checks

04

Skills

Know-how captured once, installs in any compatible harness

05

Code packages

Workflows built from known, scanned, pinned parts

06

Multi-vendor API layer

Changing models is a config change, not a rewrite

07

Stored evals

How you know a swap worked, and spot drift

= AGNOSTIC

The platform every use case runs on

Kinvecta

FOUNDATION 06 · API LAYER

A model switch should be a one-line change

Endpoint in config

Vendor API, cloud provider or approved platform

Model version pinned

A named version, never “latest”

Vendor-neutral schema

Prompts, tool calls and outputs keep the same shape



Kinvecta

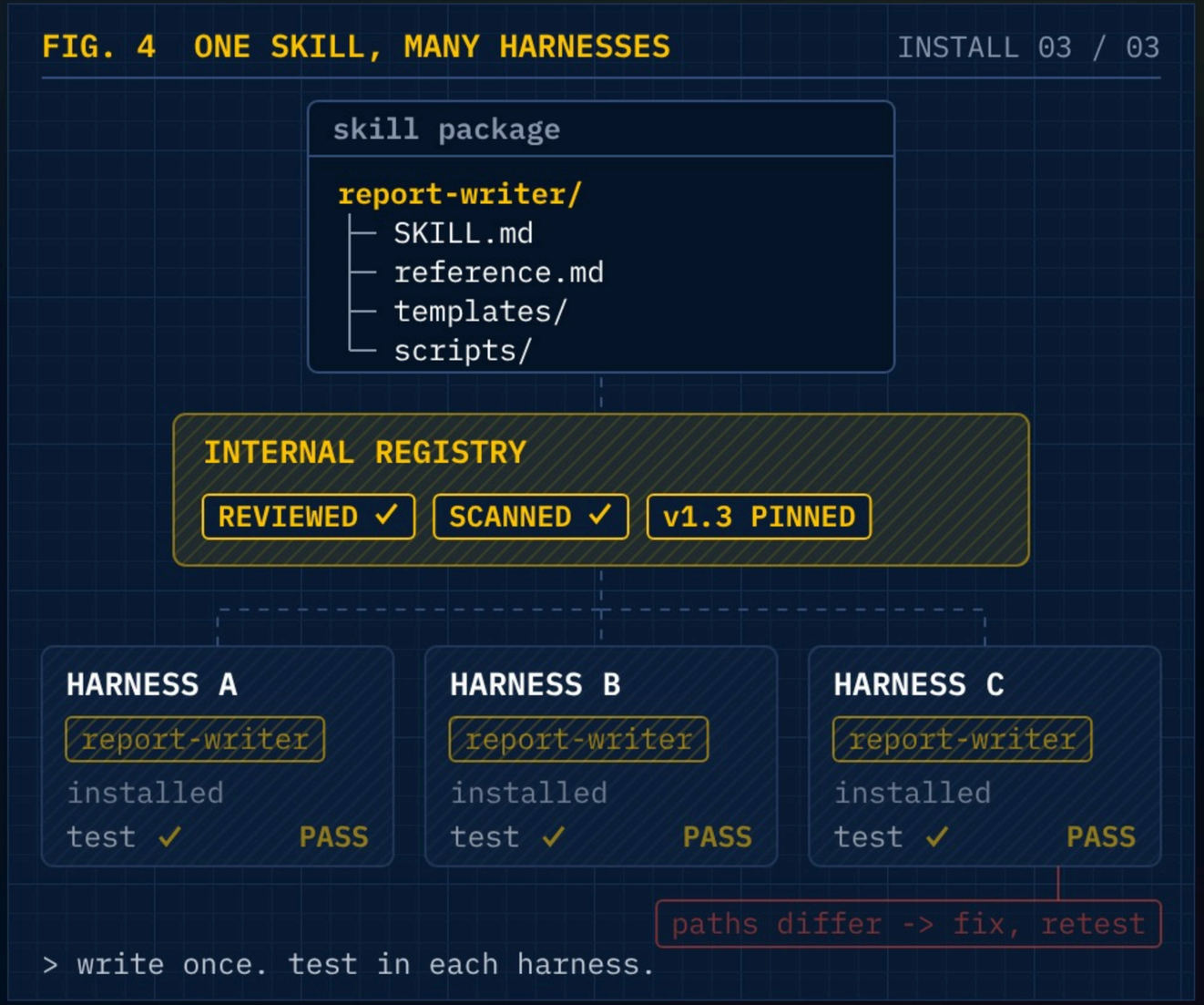
FOUNDATION 04 · SKILLS

Write a skill once, install it where you're authorized

Open format
SKILL.md folders that most major harnesses now read

Internal registry
Reviewed, scanned and version-pinned before anyone installs it

Test in each harness
Scripts, file paths and tool names can differ



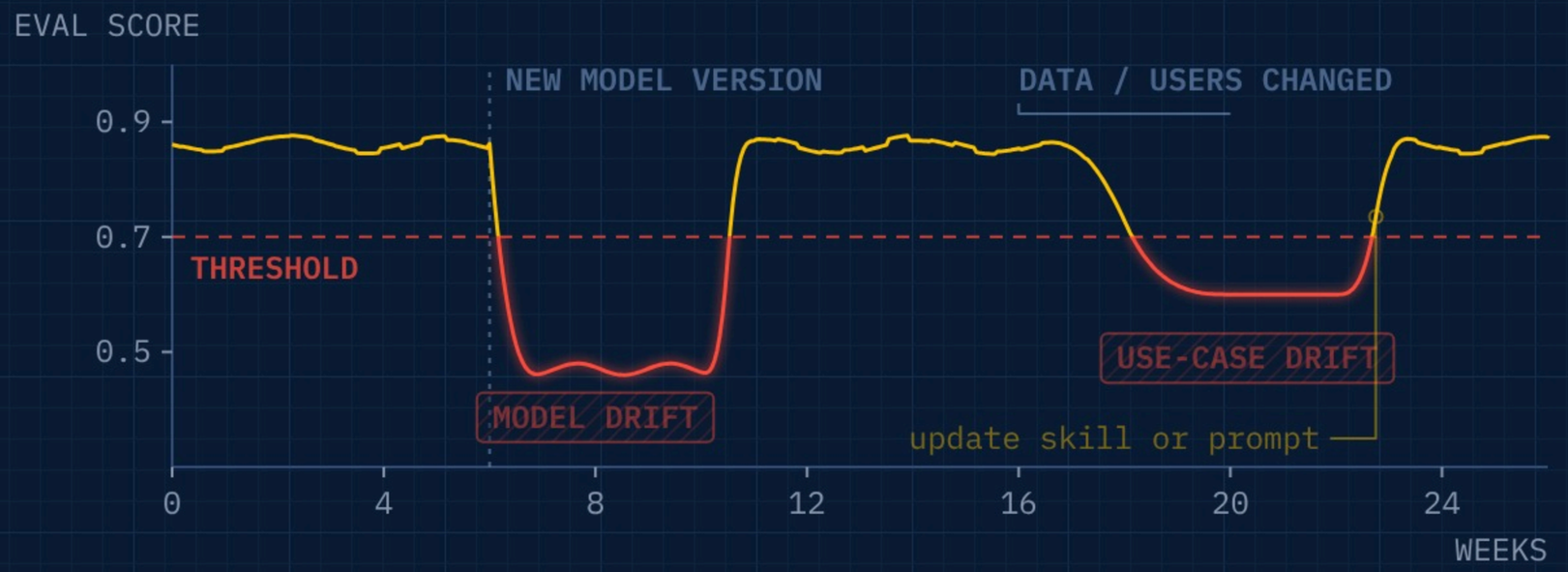
Kinvecta

FOUNDATION 07 · STORED EVALS

Evals are how you know a switch worked

FIG. 5 EVALS CATCH DRIFT

WEEK 26



STORED EVALS

SCORE	0.88
MODEL DRIFT pinned in config	FIXED
USE-CASE DRIFT skill updated	FIXED

> without evals, agnostic is a hope.

Model drift: a new version answers differently

Use-case drift: the data or users move, the prompt stays

Kinvecta

THE RULE OF THUMB

Rent the model.
Own the connections.
Own the know-how.
Own the proof.

Connections are your MCP tools and API layer. Know-how is your skills and prompts. Proof is your stored evals.

Kinvecta

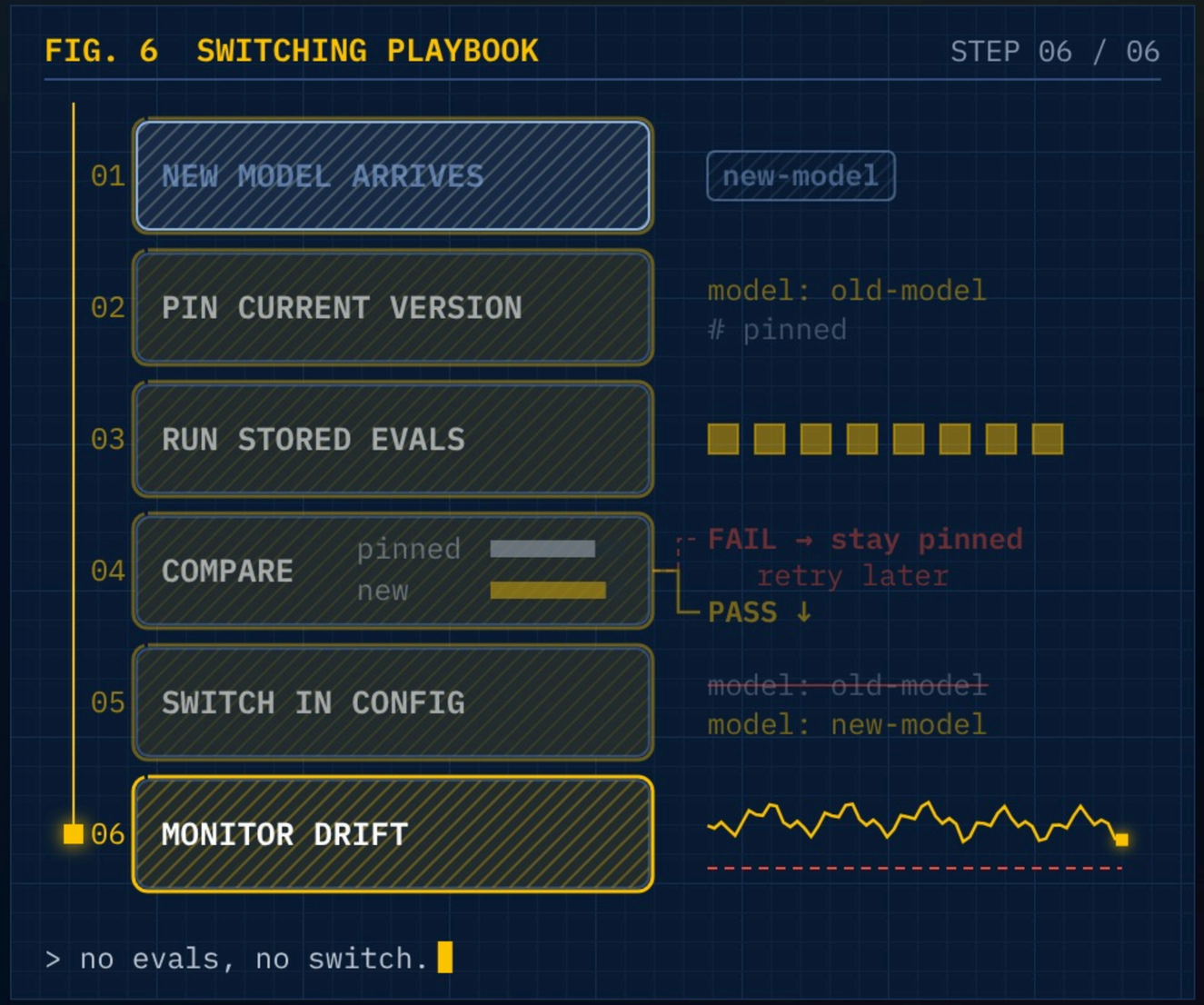
SWITCHING PLAYBOOK

When a new model arrives, test before you switch

Pin first
The current version keeps running while you test

Same evals
Run the stored test set on the new model and compare

Switch in config
Then keep monitoring for drift



Kinvecta

EXERCISE

Run a lock-in audit

Pause & try

A Score one real workflow

01 MCP tools	OWNED	VENDOR	MISSING
02 Containers	OWNED	VENDOR	MISSING
03 Scanning	OWNED	VENDOR	MISSING
04 Skills	OWNED	VENDOR	MISSING
05 Code packages	OWNED	VENDOR	MISSING
06 API layer + config	OWNED	VENDOR	MISSING
07 Stored evals	OWNED	VENDOR	MISSING

B Scenario

The model behind this workflow is retired in 30 days.

1. Where is the model named: config or code?
2. Which evals would prove the new one works?
3. Which prompts or skills may need changes?
4. Who approves the switch?
5. What would make the next one easier?

Kinvecta

CHECK FOR UNDERSTANDING

Can every learner answer these?

1

In one sentence, what makes an organization agnostic?

2

Name four of the seven foundations.

3

Why do stored evals matter most when you switch?

4

What is the difference between model drift and use-case drift?

5

Where should the model name and endpoint live?

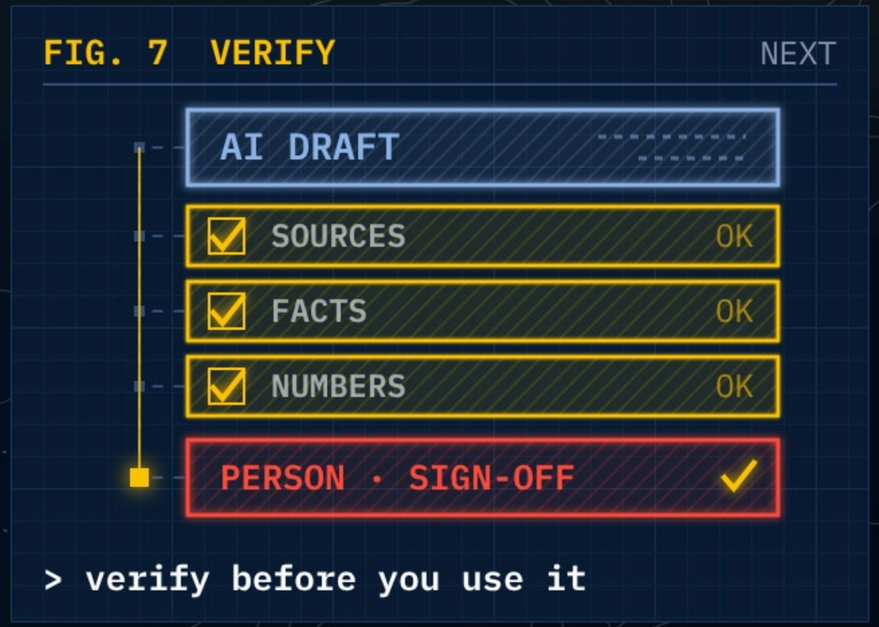
6

Why test a skill in each harness if the format is shared?

Kinvecta

NEXT · SESSION 3 · SAFE PROMPTING AND VERIFICATION

Output is fast and fluent. How do you know it is right?



What Session 3 covers

- The one rule
- Prompt like a briefing
- Saved instructions
- Verify before use